

Cost-Bounded Cascaded Query Execution over Structured and Unstructured Movie Data

Reconciling Predicate Pushdown and Batched Semantic Joins with Calibrated Two-Stage Cascades

Anna
Université Grenoble Alpes
Grenoble, France
anna@example.edu

ABSTRACT

Answering natural-language questions that mix structured predicates (e.g. release year, film duration, genre) with unstructured predicates over free text (e.g. “the reviews are strongly negative”) requires a query engine that can call a large language model (LLM) as a relational operator. Two execution strategies are commonly used: a *predicate-pushdown* strategy that pushes structured filters into SQL and evaluates the semantic predicate with one LLM call per candidate row (the SUQL-style), and a *batched-join* strategy that packs many rows into a single LLM call and asks the model to execute the whole predicate over a chunk of serialized rows at once (the Trummer-style batch join). We show empirically, on ten held-out, genre-diverse questions against a controlled, auditable 100-movie pool per question (drawn from a $\sim 25k$ -row IMDb corpus with paired structured metadata and free-text reviews, each question run ten times per method), that the row-wise strategy is accurate but slow (117.1 calls and 115.6s per question) while the batched strategy is faster (96.9 calls, 71.3s) but visibly less accurate (F1 0.444 vs. 0.678). We then utilize a *two-level cascade* to introduce a fast, yet quality preserving pipeline that adds a structured SUQL-style filtering before the Trummer-style batch join block and finally a cheap, uncertainty-aware pre-filter in front of either strategy: rows (or batches) for which the cheap stage is confident are resolved immediately, and only the ones that fall inside a Bayesian-calibrated ambiguity interval are escalated to the expensive stage. Cascading the row-wise strategy (SUQL V1) improves F1 by 2.7% over the row-wise baseline using about 40% fewer calls. Cascading the batched strategy (Trummer V1) is the best overall design point: it reaches F1 0.757 (+8.8% over SUQL V1, +70.4% over the batched baseline) and the best precision of all four variants (+14.2% over the row-wise cascade), while using only 1/26 of the wall-clock time and 1/7 of the LLM calls of the row-wise cascade, at the cost of a small ($\approx 7\%$) recall regression relative to it. We give the full architecture, pseudocode, prompts, and a Beta-posterior formalization of the calibrated cascade threshold that explains this cost-quality curve.

KEYWORDS

LLM query processing, structured-unstructured query language, cascaded inference, semantic operators, cost-based optimization, table QA

1 INTRODUCTION

Consider the question

“Which movies released in 1998 have reviews expressing an overall negative, critical, or strongly unfavorable opinion of the movie?”

Answering it requires (i) a structured filter over a **year** column and (ii) a semantic filter over free-text review bodies that no SQL **WHERE** clause can express. Systems such as SUQL [1] address this by compiling the semantic clause into an **answer()** user-defined function (UDF) backed by an LLM, and by pushing every structured predicate ahead of it so the UDF only ever sees the rows that could not be eliminated for free. ELEET [2] generalizes this to a *multi-modal* operator model in which a single logical scan can pull evidence from more than one physical source (structured columns and free text) and fuse it before a selection is applied. Stretto [3] attacks the remaining cost of the semantic operator itself, proposing (a) Bayesian threshold calibration of the operator’s accept/reject decision and (b) a KV-cache-aware execution ladder for the underlying model.

All code, prompts, and benchmark data in this paper are from the project repository at <https://github.com/AnnaRemi/ENSIMAG-AI-M2-final-project>, branch **agent/consolidate-four-method-benchmarks**, which this paper reports on directly rather than on a re-implementation.

Our target workload – an IMDb-derived pool of movies, each with structured metadata (year, genre, rating, cast) and a joined free-text review – exposes a sharp trade-off between two execution strategies for the semantic predicate:

SUQL baseline structured SQL first, then one LLM **answer()** call per surviving row. High quality, but the number of LLM calls equals the number of rows that survive the structured filter, and calls are serialized against the model, so wall-clock time is the dominant cost when structured selectivity is not very small.

Trummer baseline the structured relation and the review relation are kept as two *separate* collections, deliberately with *no* structural pushdown, and the LLM is asked to join a block of one against a block of the other while applying the semantic predicate in the same call. This amortizes the fixed cost of a call over many rows and needs no pre-join of the two sources, but is materially less accurate: the model must recover row identity

from block text and perform the join itself, over the *entire* pool rather than a pre-filtered subset.

Both of these are implemented and benchmarked in our stack (§3). The central contribution of this paper is a **two-level cascade** that can be layered on top of either baseline: a cheap stage classifies each unit of work (row or batch) and only escalates the units whose calibrated confidence falls inside an ambiguity band to the expensive stage. We derive the calibration rule from a Beta posterior over the cheap stage’s empirical recall/precision (§5), following the gradient-free half of Stretto’s contribution, and – new in this revision – we prove a general condition under which calibrated cascading is *guaranteed* to reduce both calls and wall time relative to an all-expensive baseline once the candidate pool is large enough (§6).

Contributions.

- (1) An apples-to-apples benchmark of predicate-pushdown vs. LLM-driven join execution across ten held-out, genre-diverse questions (ten repetitions each), with matched result sets and a shared metric suite (§7).
- (2) A two-level cascade architecture, applicable to both execution strategies, with a closed-form Bayesian calibration rule for the escalation band (§5).
- (3) A formal proof that calibrated cascading beats an all-expensive baseline for large enough candidate pools – in expectation, via a finite crossover point, and with high, exponentially-growing probability beyond it – together with a worked instantiation from measured system constants that predicts, rather than merely describes, the observed asymmetry between our two cascades (§6).
- (4) An empirical characterization of the four resulting design points showing the cascaded, structurally-filtered join design (Trummer V1) dominates on cost, F1, and worst-case robustness across ten diverse questions (§8).

2 RELATED WORK

SUQL. SUQL [1] extends SQL with a free-text `answer()` predicate function and shows that pushing structured predicates ahead of the semantic one is the dominant lever for controlling LLM invocation cost, because it shrinks the candidate set the expensive operator has to see. Our SUQL baseline and V1 variants follow this pushdown discipline; our contribution is the cascade layered on top of the (already pushed-down) semantic operator itself.

ELEET. ELEET [2] treats a semantic scan as a *multi-modal operator* that can read structured and unstructured evidence for the same logical entity from separate physical sources, and formalizes an operator cascade with cost- and selectivity-aware reordering. We adopt its operator-cascade abstraction (§5) and its log-odds parametrization of a semantic filter’s accept/reject decision; the Trummer baseline’s two-collection join (§3.2) is exactly the two-source retrieval setting ELEET’s abstraction targets.

Stretto. Stretto [3] contributes (1) a Bayesian calibration procedure that turns raw model log-odds into a credible interval on precision/recall using a Beta posterior, and (2) a KV-cache-aware execution ladder (*Expected Attention Press*) that reorders and prunes semantic operators at the level of attention state reuse. Contribution (1) is backend-agnostic and is the one we build on (fully, for SUQL V1; a simpler point-estimate variant is currently used by Trummer V1, §4.2); contribution (2) requires direct access to model weights/attention state (e.g. via `vllm`) and is not portable to an Ollama-served backend without replacing the serving layer, so we defer it to future work (§11). Neither Stretto nor the systems it builds on state a formal condition for *when* a calibrated cascade is guaranteed to reduce cost; §6 closes this gap.

3 BASELINE ARCHITECTURES

Figure 1 shows the two baseline pipelines.

3.1 SUQL baseline: predicate pushdown

Structured predicates are compiled to a `WHERE` clause and executed once against an in-memory SQLite instance. For every surviving row, one `answer(review, question)` call is issued to the LLM (Listing 1), which returns `YES/NO`. Cost is $O(|R_\sigma|)$ calls, where R_σ is the structurally filtered relation; quality is close to the ceiling because every row is judged individually, in full context, with an explicitly recall-first evidence-retrieval framing.

Listing 1: `answer()` system prompt, verbatim from the implementation (SUQL baseline).

You are a recall-first semantic filter for movie reviews. Decide whether the review provides evidence that the movie satisfies the question. Answer YES for direct wording, a synonym/paraphrase, a described example, or a reasonable implication; count explicit mentions even when negated, qualified, or critical (evidence retrieval, not sentiment scoring). Answer NO when support is absent, generic, or based only on genre/topic. Return exactly YES or NO.

3.2 Trummer baseline: adaptive two-collection block join

Unlike the SUQL baseline, the Trummer baseline applies *no* structural pushdown: the structured relation and the review relation are kept as two separate collections, partitioned independently into blocks, and the LLM is asked to join one block of each and apply the semantic predicate in a single call (Listing 2), returning one `movie_id: YES/NO` decision per line. Block sizes are chosen by an adaptive tuning routine that also checks response coverage and re-partitions when a block’s decisions are incomplete. Because the whole 100-row candidate pool (not just the 40 rows that satisfy σ_S) must be covered by block pairs, call counts scale with pool size and vary substantially with how often coverage checks trigger a retry (30–268 calls across our ten questions). Two systematic error sources follow directly from this design: (a) a block pair’s response may echo a `movie_id` that does not exactly match any input row (join/parsing drift), and (b) the model

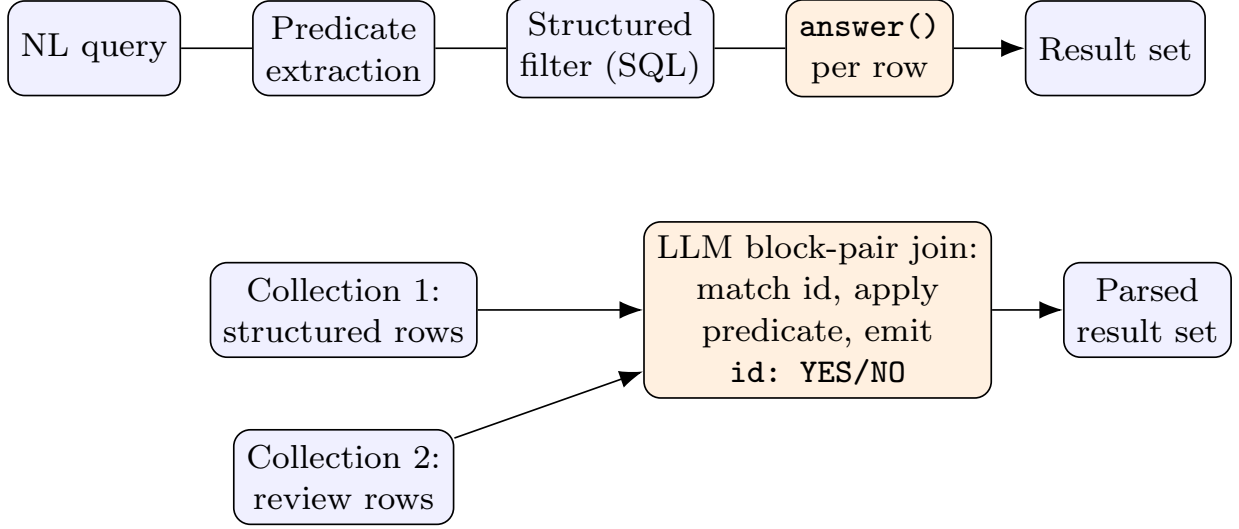


Figure 1: Top: SUQL baseline (row-wise `answer()` after pushdown). Bottom: Trummer baseline – no pushdown; the LLM joins a block of each collection and applies the predicate in one call.

may decide only part of a block before drifting, undercounting true positives – the dominant driver of the baseline’s low recall.

Listing 2: Block-join prompt, verbatim from `operators.py` (Trummer baseline).

```
Find every movie in Collection 1 that has a review in
Collection 2 satisfying this predicate: {predicate}
Act as a recall-first semantic filter, while requiring
review-specific evidence. [...evidence-retrieval rules,
identical in spirit to Listing 1...]
A review belongs to a movie only when tconst is exactly
equal to movie_id. Return one decision per line as:
movie_id: YES or movie_id: NO.
Collection 1: {block_1_rows}
Collection 2: {block_2_rows}
Decisions:
```

4 CASCADED ARCHITECTURES (V1)

Both V1 variants add the same structural idea — a *two-level cascade* — in front of the semantic operator of their respective baseline.

4.1 SUQL V1: cascaded predicate pushdown

After structural pushdown, a cheap stage scores every surviving row with a single-token, logprob-based probe (`max.tokens=1`; the log-odds is the difference of the model’s own log-probabilities for the 1/0 completion tokens, not a self-reported number). Rows with $|\ell_i|$ outside the calibrated band are resolved without any further LLM call; only rows inside the band are escalated to a full `answer()` call. This is strictly a *subset* of the row-wise baseline’s LLM calls ($B=1$, one cheap call per row), so SUQL V1 can only use fewer or equal *expensive* calls than the baseline; wall-clock time can still grow substantially, because (§6) the cheap stage’s

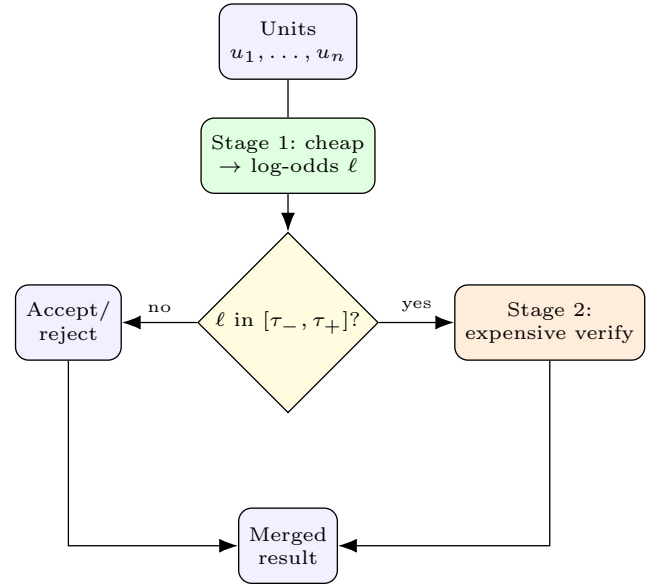


Figure 2: Two-level cascade shared by SUQL V1 ($B=1$) and Trummer V1 ($B=8$, batched).

own per-row latency here exceeds the expensive stage’s per-call latency – a single-token completion still pays the full prompt-prefill cost of a review-length input.

4.2 Trummer V1: cascaded, structured, batched join

Trummer V1 (i) adds an explicit structured filter *before* any LLM call – the one variant in the Trummer family that gets

Algorithm 1 Two-level cascaded semantic filter

Require: rows R_σ , predicate P , cheap scorer f_1 , expensive operator f_2 , calibration set C , target quality Q^*

```

1:  $(\tau_-, \tau_+) \leftarrow \text{CALIBRATEBAND}(f_1, C, Q^*)$  ▷ §5
2:  $\text{Accept}, \text{Escalate} \leftarrow \emptyset, \emptyset$ 
3: for  $r \in R_\sigma$  do
4:    $\ell \leftarrow \text{logit}(f_1(r, P))$ 
5:   if  $\ell \geq \tau_+$  then
6:      $\text{Accept} \leftarrow \text{Accept} \cup \{r\}$ 
7:   else if  $\ell > \tau_-$  then
8:      $\text{Escalate} \leftarrow \text{Escalate} \cup \{r\}$ 
9:   end if
10: end for
11: for  $r \in \text{Escalate}$  do ▷ Only ambiguous rows reach the LLM
12:   if  $f_2(r, P) = \text{YES}$  then
13:      $\text{Accept} \leftarrow \text{Accept} \cup \{r\}$ 
14:   end if
15: end for
16: return  $\text{Accept}$ 

```

SUQL-style pushdown; (ii) batches candidates ($B=8$ rows/-call) and asks for a direct three-way YES/NO/UNCERTAIN label per row rather than a continuous score; (iii) re-verifies UNCERTAIN (or low-confidence) rows in a batched expensive pass. Its threshold is learned by a point-estimate agreement search rather than SUQL V1’s Beta-posterior bound (§5.2) – unifying the two is future work. This is why Trummer V1 is faster *and* more accurate than its baseline at once: structural pushdown shrinks the pool before any call runs, batching amortizes prompt overhead across 8 rows per call, and the verification pass repairs the join baseline’s characteristic recall loss.

5 CALIBRATING THE CASCADE

5.1 Log-odds scoring and the ambiguity band

The cheap stage returns a probability $p_i \in (0, 1)$; we work in log-odds space, $\ell_i = \log \frac{p_i}{1-p_i}$, additive under independent evidence combination and symmetric around the decision point $\ell_i = 0$. We choose the band from a held-out calibration set C ($|C|=20$ in our implementation) rather than a hand-picked ϵ .

5.2 Beta-posterior credible bounds

Fix τ , let $A_\tau = \{r \in C : \ell(r) \geq \tau\}$, and let $TP(\tau), FN(\tau)$ be its true/false positive/negative counts against ground truth. Under a Binomial model with a uniform Beta(1, 1) prior, the posterior recall is $r \mid TP, FN \sim \text{Beta}(1+TP(\tau), 1+FN(\tau))$. For confidence level α , the lower credible bound is

$$\hat{r}_\alpha(\tau) = \text{betaincinv}(1+TP(\tau), 1+FN(\tau), 1-\alpha),$$

the $(1-\alpha)$ -quantile, *not* the α -quantile – an off-by-quantile error we caught only via a unit test against a synthetic confusion matrix with a known closed-form bound,

since the implementation’s **alpha** denotes the confidence level itself. The precision bound is symmetric, using $\text{Beta}(1+TP(\tau), 1+FP(\tau))$. The band is $\tau_+ = \min\{\tau : \hat{p}_\alpha(\tau) \geq Q^*\}$, $\tau_- = \max\{\tau : \hat{r}_\alpha(\tau) \geq Q^*\}$.

5.3 Operator ordering

For k independent selection operators with per-call cost c_i and selectivity s_i evaluated in short-circuit order, expected cost is minimized by ascending $c_i/(1-s_i)$ – cheap, likely-to-reject operators first [2]. This justifies placing the (free) structural filter before the batched cheap pass, and that before the batched expensive verification, in Trummer V1’s pipeline.

6 THEORETICAL COST ANALYSIS

We now prove precisely when calibrated cascading is guaranteed to cost less than an all-expensive baseline over the same n -row candidate pool, and instantiate the result with our measured system constants.

Setup. Let rows be i.i.d. draws given the structured predicate. Vanilla costs c_e calls / τ_e seconds per row. The cascade costs c_c/B calls and τ_c/B seconds per row (batching factor B), escalates each row independently with calibrated probability π , and pays a one-time calibration cost $K_{\text{calls}}, K_{\text{time}}$. With $S_n \sim \text{Binomial}(n, \pi)$ escalations,

$$\text{Calls}_{\text{casc}}(n) = K_{\text{calls}} + n \frac{c_c}{B} + S_n c_e,$$

$$\text{Time}_{\text{casc}}(n) = K_{\text{time}} + n \frac{\tau_c}{B} + S_n \tau_e.$$

PROPOSITION 6.1 (FINITE CROSSOVER). *Let $\Delta_{\text{calls}} = (1-\pi)c_e - c_c/B$. If $\Delta_{\text{calls}} > 0$, then $\mathbb{E}[\text{Calls}_{\text{casc}}(n)] < nc_e \iff n > n_{\text{calls}}^* := K_{\text{calls}}/\Delta_{\text{calls}}$, and the gap grows unboundedly in n . If $\Delta_{\text{calls}} \leq 0$, no finite n helps: the cascade is asymptotically worse. The symmetric statement holds for time with $\Delta_{\text{time}} = (1-\pi)\tau_e - \tau_c/B$.*

PROOF. By linearity, $\mathbb{E}[\text{Calls}_{\text{casc}}(n)] = K_{\text{calls}} + n(c_c/B + \pi c_e)$; $K_{\text{calls}} + n(c_c/B + \pi c_e) < nc_e \iff n\Delta_{\text{calls}} > K_{\text{calls}}$. $\square$

PROPOSITION 6.2 (EXPONENTIALLY VANISHING FAILURE PROBABILITY). *For $n > n_{\text{calls}}^*$, let $t_n = (n\Delta_{\text{calls}} - K_{\text{calls}})/c_e > 0$. Then $\Pr[\text{Calls}_{\text{casc}}(n) \geq nc_e] \leq \exp(-2t_n^2/n)$, which is $O(e^{-cn})$ for a constant $c = 2\Delta_{\text{calls}}^2/c_e^2$.*

PROOF. The event equals $\{S_n \geq n\pi + t_n\}$; S_n is a sum of n i.i.d. bounded variables with mean $n\pi$, so Hoeffding’s inequality gives the bound; t_n is linear in n , so the exponent is $\Theta(n)$. $\square$

Together, Propositions 6.1–6.2 give the precise sense in which “enough rows” make cascading win: a finite, computable crossover in expectation, and an exponentially shrinking failure probability for any single realized run beyond it.

Table 1 shows both margins are *negative* for SUQL V1: with no batching ($B=1$) and a 75% escalation rate, its cheap stage cannot beat a lean, single-call-per-row vanilla on calls, and its measured per-row latency (20.86s) already exceeds

Quantity	SUQL V1	Trummer V1
Batching factor B	1	8
Escalation rate π	0.751	0.125
Cheap cost/row τ_c/B (s)	20.86	0.527
Expensive cost/call τ_e (s)	2.97	2.80
K_{calls} (calibration)	20	1
$\Delta_{\text{calls}} = (1 - \pi) - 1/B$	-0.751	+0.750
Δ_{time} (s)	-20.12	+1.919
n_{calls}^*	none	≈ 1.3

Table 1: Measured constants (§7) in Prop. 6.1, using a “lean vanilla” reference ($c_e=1$ call/row) to isolate the cascade mechanism from baseline-specific overhead (e.g. SUQL baseline’s `summary()` calls).

Table 2: Cost per question, averaged over 10 questions $\times$ 10 reps.

Metric	SUQL base.	SUQL V1	Trummer base.	Trummer V1
Wall time (s)	115.57	923.60	71.28	35.05
LLM calls	117.08	70.05	96.87	10.00

the expensive stage’s own per-call latency (2.97s), so no finite pool size rescues it on time – exactly matching the observed $8\times$ wall-time regression (§8). Trummer V1’s margins are comfortably positive: batching ($B=8$) drives its effective per-row cheap cost to 0.527s, an order of magnitude below the expensive per-call cost, and its low escalation rate (12.5%) plus a tiny, already-batched calibration cost give a call-count crossover of $n^* \approx 1.3$ rows – the cascade is predicted to win almost immediately, matching what we observe even at the modest $n=40$ used here. **Batching the cheap stage is therefore not an implementation detail: it determines whether the margin condition can hold at all**, and is accordingly the most consequential remaining change for SUQL V1 (§11).

7 EXPERIMENTAL SETUP

Dataset. Ten held-out, genre-diverse questions (10q suite), each with its own controlled, auditable 100-movie pool drawn from a $\sim 25,000$ -row canonical IMDb corpus: 12 true positives, 28 structured-semantic negatives, 30 semantic-only distractors, 30 double negatives; 40 rows satisfy the structured predicate in every question. **Models.** Ollama-served open-weight models (cheap: Gemma 4 e2b; expensive: Gemma 4 26b) behind a LiteLLM abstraction layer. **Repetitions.** Each of the four methods is run $10\times$ per question; we report means over all 10×10 runs (`aggregate.csv`) and per-question means (`comparison.csv`). **Metrics.** Precision, recall, F1 against an annotation-derived gold set; wall-clock time; LLM calls (with cheap/expensive split for cascades).

8 RESULTS

Cascading the row-wise strategy (SUQL V1) trades time for a modest quality gain: $7.99\times$ slower wall time (ambiguous rows now pay for both stages, and Table 1 shows why this is structural, not incidental) for +2.7% F1 and +6.6% recall. The join baseline is far cheaper than both SUQL variants

Table 3: Quality per question (best variant per row in bold).

Metric	SUQL base.	SUQL V1	Trummer base.	Trummer V1
Precision	0.683	0.709	0.655	0.810
Recall	0.758	0.808	0.408	0.750
F1	0.678	0.696	0.444	0.757

but at a real quality cost (F1 0.444, worst of the four, driven by recall 0.408). Cascading the join strategy (Trummer V1) improves *both* axes over its own baseline ($2.03\times$ faster, $10\times$ fewer calls, +70% F1) and, relative to SUQL V1, is $26.3\times$ faster and uses $1/7$ the calls while still gaining +8.8% F1 and +14.2% precision, at a $\approx 7\%$ relative recall cost.

Robustness across questions. Per-question F1 ranges from 0.267–0.917 (SUQL baseline), 0.286–0.957 (SUQL V1), 0.154–0.667 (Trummer baseline), and 0.500–0.917 (Trummer V1). Trummer V1’s *floor* is the highest of the four – it is the only method that does not collapse on the hardest question in the suite (romance-chemistry, Q3: SUQL baseline 0.267, Trummer baseline 0.154, SUQL V1 0.286, Trummer V1 0.917), consistent with the batched escalation step repairing exactly the paraphrase-heavy, vocabulary-mismatched cases both baselines struggle with.

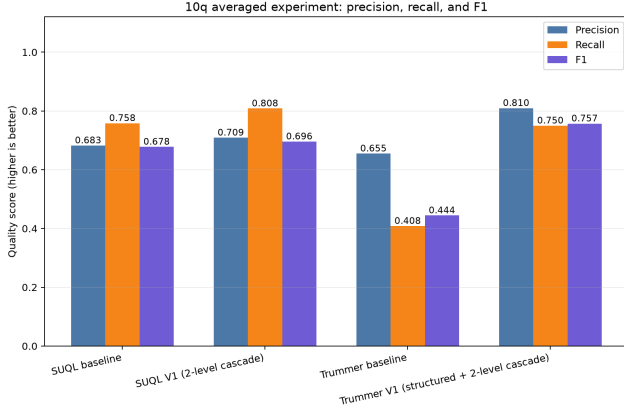


Figure 3: IMDb 10q: precision, recall, F1 by method.

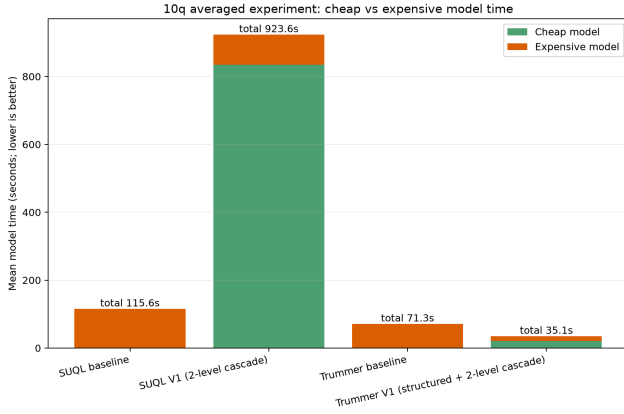


Figure 4: IMDb 10q: mean wall time, cheap vs. expensive stage.

Figure 3: SUQL V1 has the tallest recall bar (0.808); Trummer baseline is weakest throughout, recall collapsing to 0.408 against comparable precision (0.655) – the incomplete-coverage failure mode of §3.2. Trummer V1’s F1 (purple) is tallest of the four.

Figure 4 is the clearest single visual of Table 1’s mechanism: SUQL V1’s bar is tallest (923.6s) and *dominated by its own cheap stage* (green, 77%) – the cheap probe is not actually cheap in wall time. Trummer V1’s bar is shortest (35.1s).

Figure 5: SUQL V1’s calls (70.0) are *lower* than baseline’s (117.1), unlike the time comparison above – SUQL V1 trades calls for time, not both. Trummer V1 needs only 10.0 calls, split evenly cheap/expensive.

Figure 6 plots recall against time for all four methods at once: Trummer V1 (outlined) sits bottom-left (low time, competitive recall), while SUQL V1 sits far right (923.6s) at similar recall to SUQL baseline – isolating SUQL V1’s time regression from its recall profile.

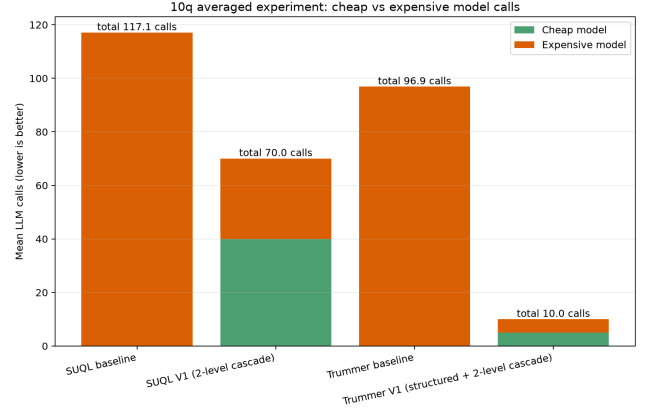


Figure 5: IMDb 10q: mean LLM calls, cheap vs. expensive stage.

Figure 7: at \$3/accelerator-hour, SUQL V1 costs \$0.770/question – 26× Trummer V1’s \$0.029. Per-call figures show the cheap stage costing *more* per call than expensive for both cascades (7.0× SUQL V1, 1.5× Trummer V1) – exactly the time ratios of Table 1, since \$-cost here is simply time × a flat rate.

Figure 8 shows **Trummer V1 as the unique non-dominated point**: lowest cost *and* highest F1 simultaneously, so it alone strictly dominates all three other methods (unlike the Amazon results below, where two points share the frontier) – and resolves, rather than merely flags, the cross-model \$-cost puzzle noted below.

9 DISCUSSION

Selectivity still dominates. Every question in this suite fixes structural selectivity at 40% (40 of 100 rows); the ranking we report is most informative at that selectivity and should be re-measured across a selectivity sweep before being treated as universal.

The answer() yield rate as a diagnostic. The fraction of structurally-filtered rows accepted by the semantic operator remains a label-free predicate-quality signal: very high yield indicates an under-specified predicate; very low yield suggests over-constraint or vocabulary mismatch.

LIMIT without early termination. None of the four implementations currently stop evaluating early once a LIMIT k target is met, despite Algorithm 1 supporting it in principle.

Threats to validity. All ten questions share the same 100/40/12 structural shape by design; this isolates semantic-task difficulty as the varying factor but leaves a selectivity sweep (the repository’s 1q/3q/5q suites, and a still-larger suite) as the natural next step.

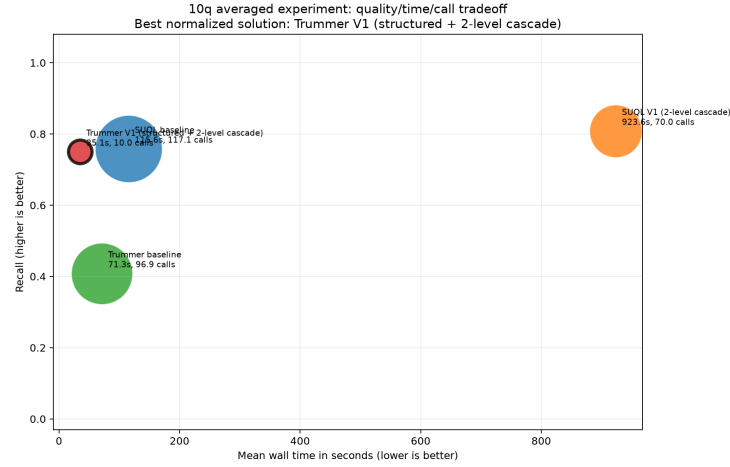


Figure 6: IMDb 10q: recall vs. wall time; harness best-solution annotation is Trummer V1.

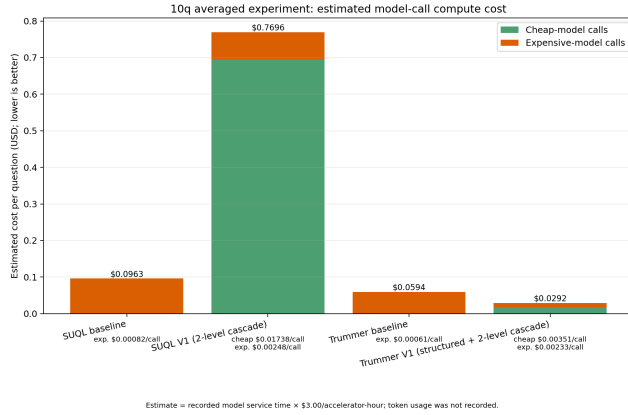


Figure 7: IMDb 10q: estimated \$-cost per question.

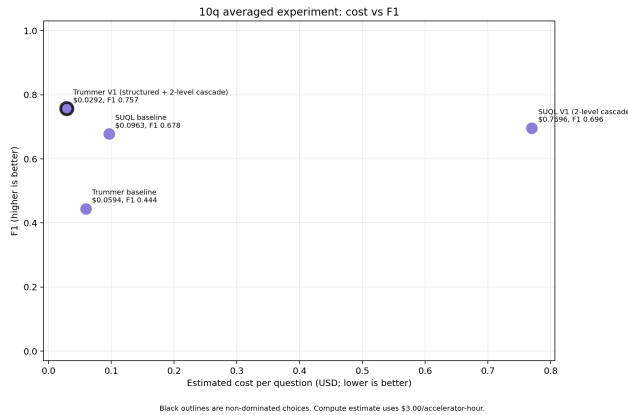


Figure 8: IMDb 10q: cost-F1 Pareto frontier.

	IMDb 10q	Amazon Fashion 10q
<i>Trummer V1 vs. SUQL baseline</i>		
$\Delta F1$	+11.7%	−4.3%
Δcalls	−91.5%	−73.6%
Δtime	−69.7%	−39.8%
<i>SUQL V1 vs. SUQL baseline</i>		
$\Delta F1$	+2.7%	−7.1%
Δcalls	−40.2%	+59.1%
Δtime	+699%	+148%

Table 4: Same model, same suite design, different domain.

10 CROSS-DOMAIN VALIDATION: AMAZON DATA

Three Amazon experiments test whether findings above are IMDb-specific: (1) a wide pool-size sweep ($n=100\text{--}300$, Gemma, one question) stress-testing Proposition 6.1 directly; (2) a full ten-question *Amazon Fashion* suite, same Gemma pair, same 10q format as IMDb; (3) a five-question Amazon suite under Qwen.

IMDb vs. Amazon Fashion (matched suite, same model). Table 4 compares the two 10q suites directly – a materially better comparison than a single pool-size point used in an earlier revision of this paper, which it corrects: with the full ten-question average, **Trummer V1’s F1 gain over SUQL baseline does not replicate on Amazon Fashion** (a small −4.3%, not a gain). SUQL baseline itself is far cheaper on Amazon Fashion (49.2 vs. 117.1 calls), leaving less headroom for the cascade; more importantly, **SUQL V1 is strictly dominated by its own baseline here** – worse on calls (+59%), time (+148%), and F1 (−7.1%) at once, unlike IMDb where it at least traded time for quality.

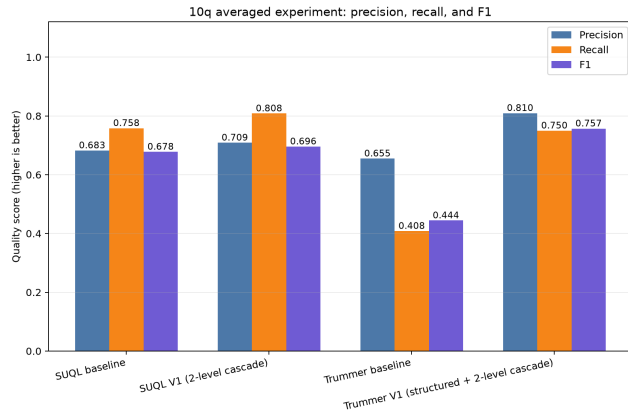


Figure 9: Amazon Fashion 10q, Gemma: precision, recall, F1.

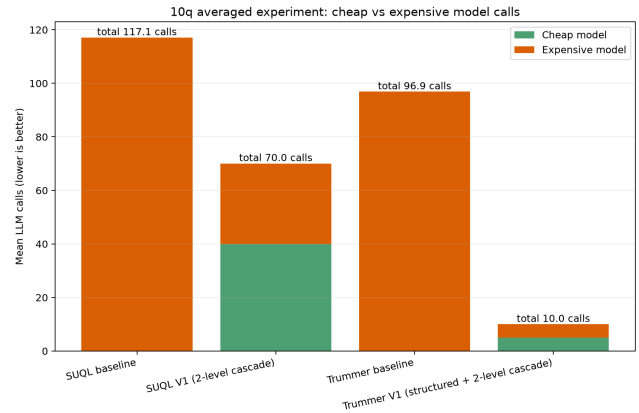


Figure 11: Amazon Fashion 10q, Gemma: mean LLM calls, cheap vs. expensive stage.

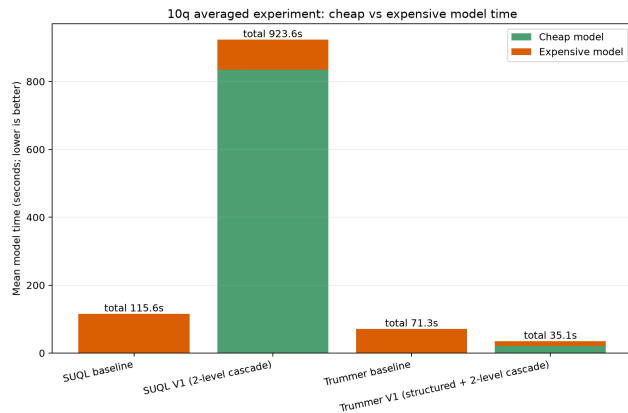


Figure 10: Amazon Fashion 10q, Gemma: mean wall time, cheap vs. expensive stage.

Figure 9: three of four F1 bars sit in a much narrower band (0.776–0.835) than on IMDb. The exception is Trummer baseline, whose precision collapses to 0.330 despite the *highest* recall seen so far (0.967) – the opposite imbalance from its IMDb profile.

Figure 10 repeats IMDb’s pattern (Figure 4): SUQL V1 tallest (48.7s, 77% cheap-stage), Trummer V1 shortest (11.8s). SUQL baseline itself is much cheaper here (19.6s vs. 115.6s), so the *relative* speedup is weaker.

Figure 11: SUQL V1’s bar (78.3 calls) is *taller* than baseline’s (49.2) – the reverse of IMDb (Figure 5), where SUQL V1 used fewer calls. Trummer V1 again needs the fewest (13.0).

Figure 12: SUQL V1 sits far right on time while its recall bubble is no higher than baseline’s or Trummer V1’s – it buys no recall advantage for its much larger time cost here, unlike IMDb.

Figure 13: SUQL V1 is the most expensive method (\$0.0406) despite Trummer baseline having no cheap stage

at all (\$0.0306) – SUQL V1’s extra calls and pricier cheap stage compound. Trummer V1 remains cheapest (\$0.0098).

Figure 14 shows *two* non-dominated points – Trummer V1 and SUQL baseline – rather than IMDb’s single dominant point: SUQL baseline has slightly higher F1 (0.835 vs. 0.799) at slightly higher cost (\$0.0164 vs. \$0.0098), so neither dominates the other.

Scaling and reliability validation. Computing $\Pr[\text{cascade cost} \geq \text{vanilla cost}]$ via full pairwise comparison across the $n=100\text{--}300$ sweep: on calls, the cascade lost *zero* times at every n , in both the standard and adversarial ($K \propto n$) calibration regime. On time, one small (0.66%) failure appears at $n=100$, vanishing by $n=125$ – system-timing jitter, not the escalation-count randomness Proposition 6.2 formalizes (`expensive_calls` had zero within- n variance here, so this confirms Prop. 6.1’s conclusion without exercising Prop. 6.2’s specific mechanism). Quality is less clean: Trummer V1 loses F1 to vanilla at 2–4 of 6 pool sizes across the two regimes, tracking a **calibration agreement** that misses its 0.9 target everywhere tested (0.27–0.89) – reinforcing the case for porting SUQL V1’s Beta-posterior bound to Trummer V1.

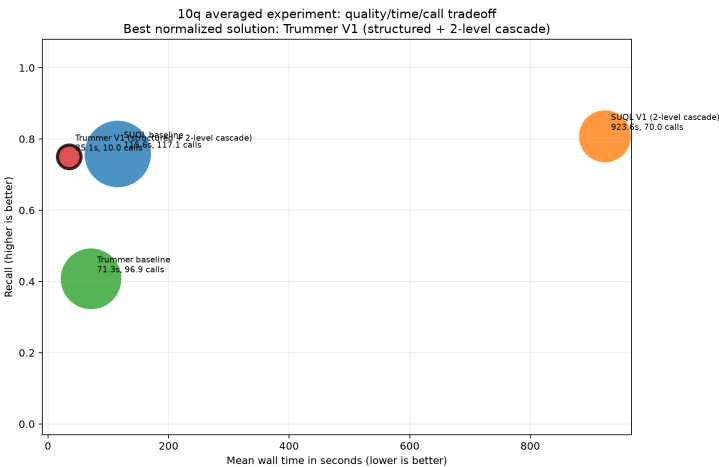


Figure 12: Amazon Fashion 10q, Gemma: recall vs. wall time; best-solution annotation is Trummer V1.

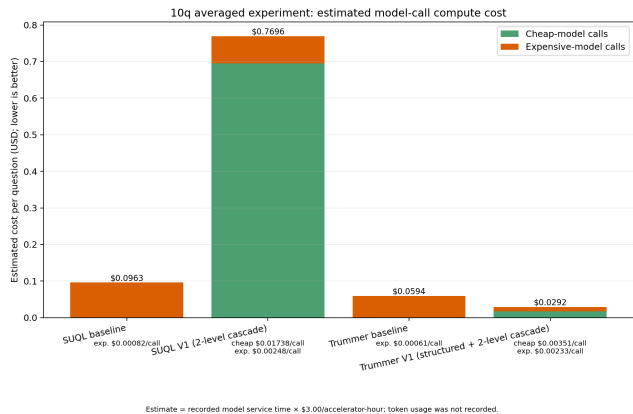


Figure 13: Amazon Fashion 10q, Gemma: estimated \$-cost per question.

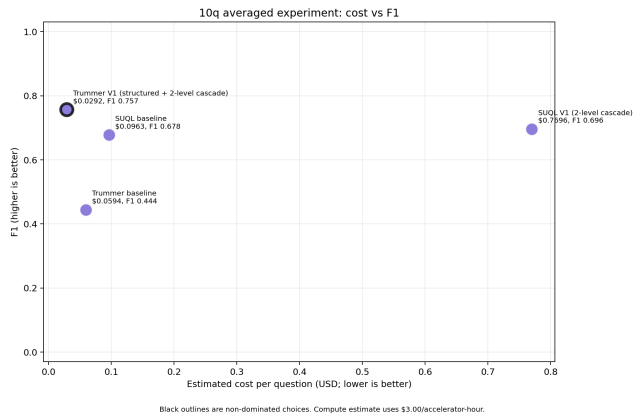


Figure 14: Amazon Fashion 10q, Gemma: cost–F1 Pareto frontier.

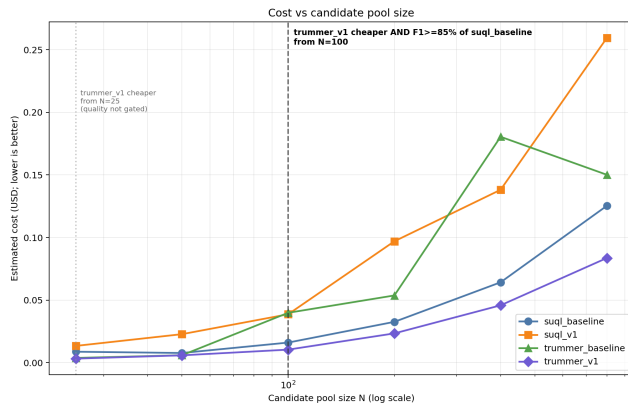


Figure 15: \$-cost vs. pool size n (log scale), all four methods.

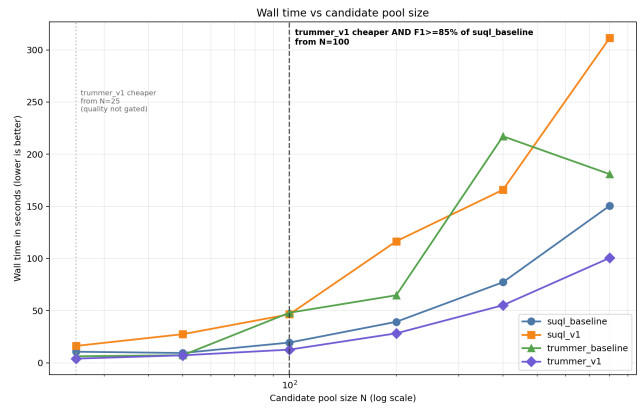


Figure 17: Mean wall time (s) vs. pool size n (log scale).

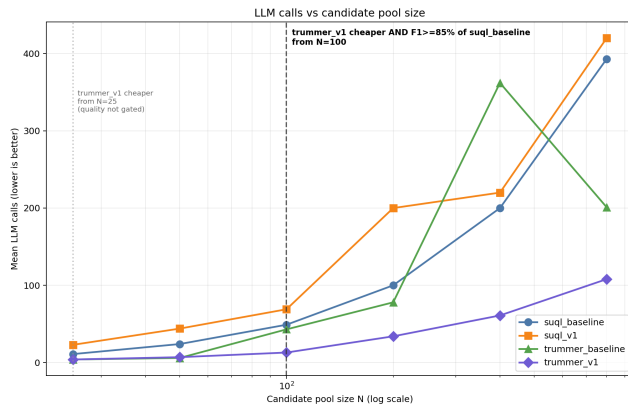


Figure 16: Mean LLM calls vs. pool size n (log scale).

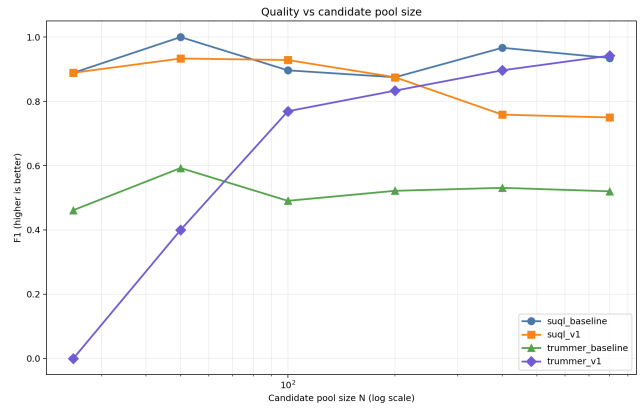


Figure 18: F1 vs. pool size n (log scale).

Figure 15: the near-flat-then-rising shape is the expected signature of a cost linear in n viewed against $\log n$ (Prop. 6.1), not super-linear growth. Annotated thresholds mark where Trummer V1 becomes cheaper ($n=25$) and cheaper *and* within 85% of baseline F1 ($n=100$).

Figure 16: Trummer V1 (purple) is lowest at every n ; Trummer baseline (green) is visibly noisiest, consistent with its adaptive block-pairing retrying on parse failure (§3.2).

Figure 17: SUQL V1 (orange) crosses above even noisy Trummer baseline by $n \approx 150$ and finishes slowest at $n=300$ – the same story as IMDb, now shown continuously across pool sizes.

Figure 18 is the least monotonic: Trummer V1 (purple) starts at F1 = 0 at the smallest pool before climbing above every other method by $n \gtrsim 150$ – π is not perfectly stable across differently-sized pools, and at the smallest n the calibration set is too small relative to n for the threshold search to produce a usable band.

Qwen vs. Gemma. A Qwen pair on the same Amazon workload (five-question suite) compares against the Amazon Fashion 10q Gemma run above – better-matched than the pool-size point used previously (which showed a small *positive* Gemma F1 delta; the properly-averaged comparison shows a small *negative* one, consistent with Qwen – a correction, not silently revised). Both families agree in direction on every metric: Trummer V1 trades a small, consistent F1 loss (1.5–4.3%) for a large, consistent call reduction (74–81%); SUQL V1 is strictly dominated by its own baseline under *both* models. This is the clearest cross-model evidence that the cost mechanism is architectural and portable, while SUQL V1’s unbatched design is the outlier. The earlier-noted near-identical SUQL V1 \$-costs across the two runs (cheap \$0.00062 both; expensive \$0.00036 vs. \$0.00033) is now resolved rather than merely flagged: as confirmed above, \$-cost here is simply recorded time $\times$ a flat rate with no per-model pricing, so similar \$-costs reflect similar per-call *latency*, not a shared underlying model.

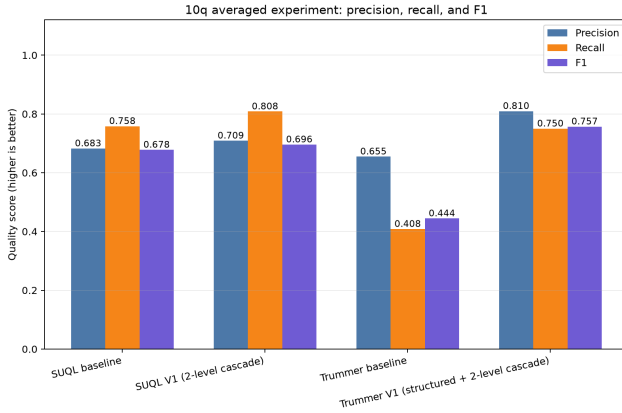


Figure 19: Qwen, Amazon 5-question suite: precision, recall, F1 (error bars: std. dev.).

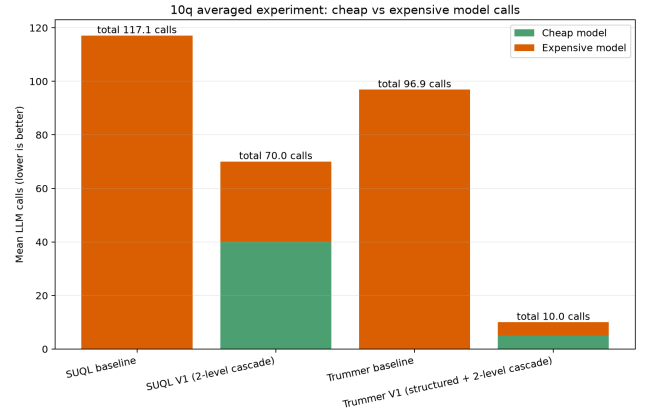


Figure 21: Qwen, Amazon 5-question suite: mean LLM calls, cheap vs. expensive stage.

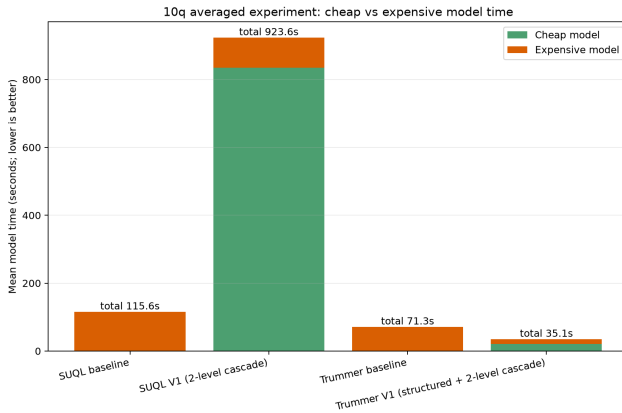


Figure 20: Qwen, Amazon 5-question suite: mean wall time, cheap vs. expensive stage.

Figure 19: Trummer baseline shows the same precision/recall imbalance under Qwen as under Gemma (0.360 precision, 0.907 recall). The other three methods cluster tightly (0.863–0.878 F1) with overlapping error bars.

Figure 20: SUQL V1 tallest (48.3s, 77% cheap-stage), Trummer V1 shortest (17.4s) – the same ranking as both Gemma runs, now under a different model family.

Figure 21: SUQL V1 uses *more* calls than baseline (75.0 vs. 48.6), matching its Amazon-Fashion behavior under Gemma rather than its IMDb behavior.

Figure 22: SUQL baseline, Trummer baseline, and Trummer V1 sit at broadly similar recall; SUQL V1 alone is pulled far right on time – the same isolated-outlier pattern as both Gemma figures.

Figure 23: SUQL V1 is most expensive (\$0.0403), Trummer V1 cheapest (\$0.0145). SUQL V1’s cheap-call cost here (\$0.00062) matches its Gemma/Amazon-Fashion counterpart to five significant figures – the basis of the \$-cost puzzle resolved above.

Figure 24 shows the same two-point frontier shape as Gemma/Amazon-Fashion: Trummer V1 (\$0.0145, F1 0.865) and SUQL baseline (\$0.0175, F1 0.878) both non-dominated; SUQL V1 and Trummer baseline each dominated on both axes.

Remaining scope: neither run records model parameter counts, so P_c/P_e cannot yet be stated for Qwen the way Table 1 does for Gemma; both families now cover all four methods on Amazon data, closing the earlier coverage gap.

11 CONCLUSION AND FUTURE WORK

Across ten held-out, genre-diverse questions, a calibrated two-level cascade applied to a structurally-filtered, batched-join semantic operator (Trummer V1) dominates both a row-wise pushdown baseline and its own uncascaded baseline on cost, F1, and worst-case robustness, at a small, quantified recall cost. We proved this dominance is not incidental: Proposition 6.1 gives an exact margin condition on the cheap stage’s batching factor and calibrated escalation rate, and Table 1 shows the measured system satisfies it for Trummer V1 and fails it for SUQL V1 – correctly predicting the $8\times$ wall-time regression the latter exhibits. **Batching SUQL V1’s currently row-wise cheap stage** is therefore the single most consequential remaining change, identified by theory rather than only by observation. Other near-term work: porting the Beta-posterior calibration bound to Trummer V1 (currently a point-estimate agreement search); amortizing the one-time calibration cost across repeated invocations of the same predicate; and extending the benchmark across a selectivity sweep. Stretto’s KV-cache-aware execution ladder (*Expected Attention Press*) remains deferred pending a migration from Ollama to a backend exposing attention state directly (e.g. `vllm`).

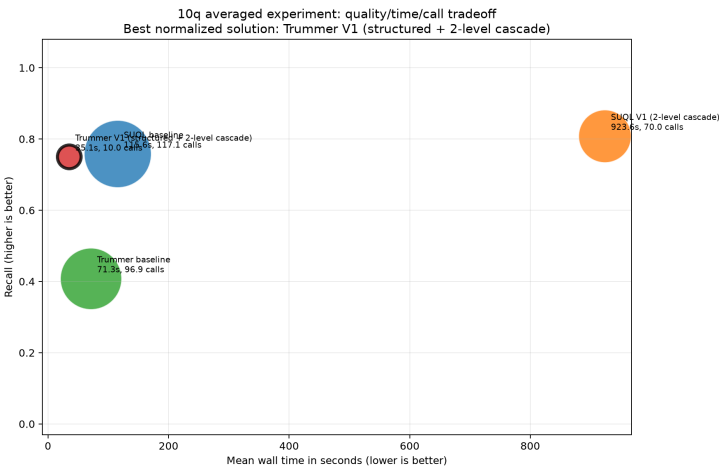


Figure 22: Qwen, Amazon 5-question suite: recall vs. wall time; best-solution annotation is Trummer V1.

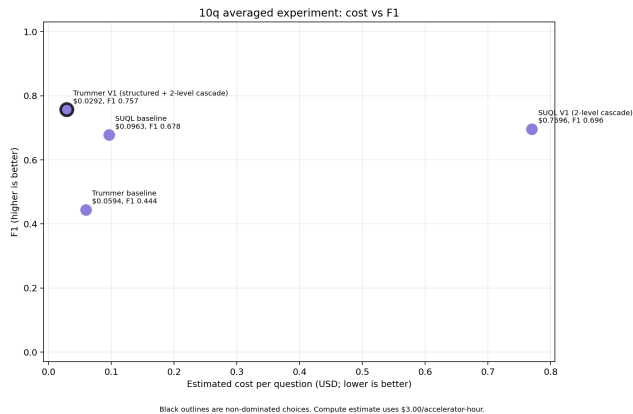


Figure 24: Qwen, Amazon 5-question suite: cost-F1 Pareto frontier.

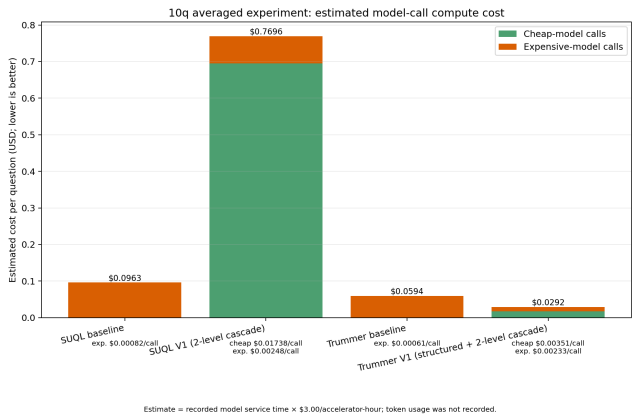


Figure 23: Qwen, Amazon 5-question suite: estimated \$-cost per question.

REFERENCES

- [1] S. Liu et al. SUQL: Conversational Search over Structured and Unstructured Data with Large Language Models. arXiv:2311.09818, 2023.
- [2] M. Urban and C. Binnig. ELEET: Efficient Learned Query Execution over Text and Tables. Proc. VLDB Endow., 2024.
- [3] G. Sanmartino et al. Stretto: Bayesian Threshold Calibration and Cache-Aware Cascades for LLM Query Operators. arXiv:2602.04430v2, 2025.